

Daily Progress Report

Week 6_Day2: Global State Management with Context API

Program	3-Month MERN Stack Internship Program
Intern Name	Mohsin Ali
Week	Week_6 (Day 2)
Date	28 July, 2026

Day 2: Global State Management with Context API

1.1 Topics Studied

- Understanding the concept of global state in React applications
- Identifying the problem of prop drilling and its impact on component communication
- Using the React Context API to share data across multiple components
- Creating a Context using createContext()
- Providing global state using the Context.Provider
- Accessing shared data using the useContext() hook
- Managing application-wide theme state (Light/Dark Mode)
- Updating global state using reusable functions inside the context
- Persisting theme preference using Local Storage
- Dynamically updating CSS variables based on the selected theme
- Applying responsive layouts while maintaining consistent theming across the application
- Organizing the project into reusable and maintainable React components

1.2 Task Description

Developed a professional **Personal Finance Dashboard** in React to demonstrate the practical implementation of global state management using the **React Context API**. The application was structured with reusable components, including a Sidebar, Navbar, Dashboard, Summary Cards, Transaction Panel, Budget Progress, Category Breakdown, and Footer. Instead of passing the theme state through multiple component levels, a centralized **Theme Context** was created to manage the application's light and dark modes globally.

The entire application was wrapped inside a **Theme Provider**, allowing every component to access the current theme and toggle function directly through the useContext hook without prop drilling. Theme changes were applied instantly across the application using CSS variables, ensuring a consistent user interface. Additionally, the selected theme was stored in Local Storage, allowing the user's preference to persist across browser refreshes and future sessions. The dashboard was designed with a responsive layout to provide an optimal viewing experience on both desktop and mobile devices.

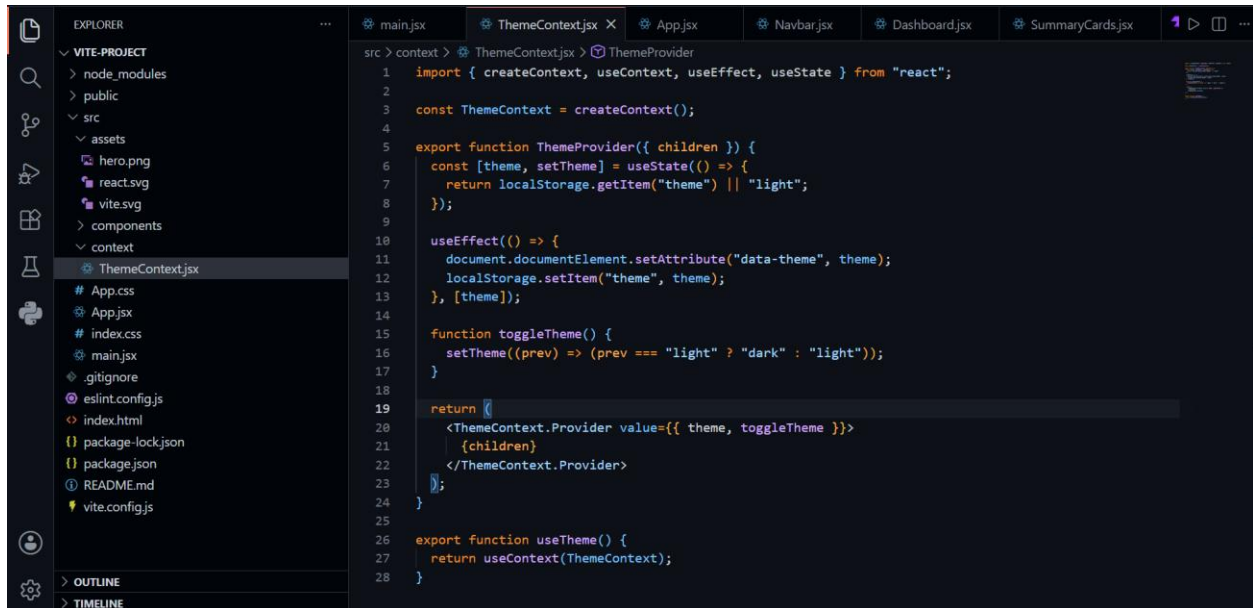
1.3 Implementation Details

- Created a reusable ThemeContext using createContext().
- Wrapped the entire React application inside a ThemeProvider.
- Managed the global theme state using the useState() hook.
- Shared the theme state and toggle functionality through the Context Provider.
- Consumed global state inside components using the useContext() hook.
- Eliminated prop drilling by providing direct access to global state across the component tree.
- Implemented a theme toggle button in the Navbar to switch between Light and Dark modes.
- Applied CSS Variables to dynamically update colors, backgrounds, text, borders, and shadows across the entire application.
- Stored the selected theme in Local Storage to preserve user preferences.
- Retrieved the saved theme during application initialization.
- Designed a responsive Personal Finance Dashboard consisting of reusable components such as Sidebar, Navbar, Summary Cards, Transactions, Budget Progress, Category Breakdown, and Footer.
- Organized the project into separate component, context, and styling files to improve maintainability and scalability.
- Implemented smooth UI transitions to enhance the user experience when switching themes.

1.4 Outcome

Successfully developed a professional **Personal Finance Dashboard** that demonstrates the practical implementation of global state management using the React Context API. The project reinforced the understanding of avoiding prop drilling through centralized state management, sharing data efficiently across multiple components using the useContext() hook, and creating reusable providers for application-wide functionality. Furthermore, implementing persistent theme management with Local Storage, dynamic styling through CSS variables, and a responsive component-based architecture strengthened best practices for building scalable, maintainable, and modern React applications.

Screenshots:

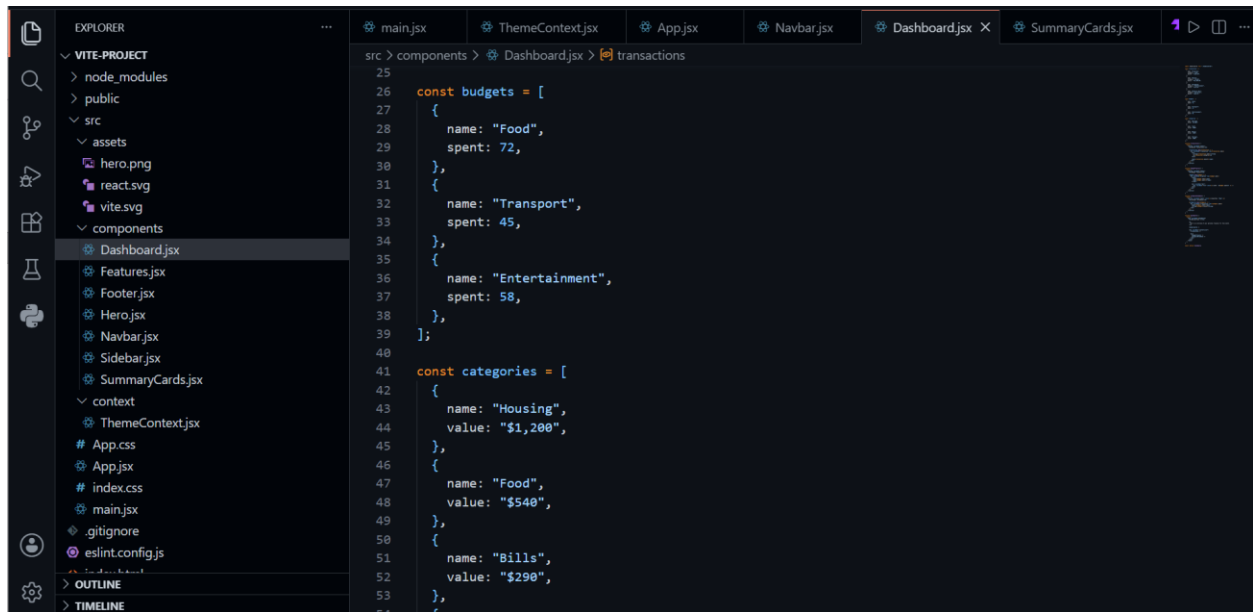


This screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor on the right. The file explorer shows the project structure with the following files and folders:

- VITE-PROJECT
 - node_modules
 - public
 - src
 - assets
 - hero.png
 - react.svg
 - vite.svg
 - components
 - context
 - ThemeContext.jsx (selected)
- OUTLINE
- TIMELINE

The code editor shows the content of ThemeContext.jsx:

```
1 import { createContext, useContext, useEffect, useState } from "react";
2
3 const ThemeContext = createContext();
4
5 export function ThemeProvider({ children }) {
6   const [theme, setTheme] = useState(() => {
7     return localStorage.getItem("theme") || "light";
8   });
9
10  useEffect(() => {
11    document.documentElement.setAttribute("data-theme", theme);
12    localStorage.setItem("theme", theme);
13  }, [theme]);
14
15  function toggleTheme() {
16    setTheme((prev) => (prev === "light" ? "dark" : "light"));
17  }
18
19  return (
20    <ThemeContext.Provider value={{ theme, toggleTheme }}>
21      {children}
22    </ThemeContext.Provider>
23  );
24 }
25
26 export function useTheme() {
27   return useContext(ThemeContext);
28 }
```



This screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor on the right. The file explorer shows the project structure with the following files and folders:

- VITE-PROJECT
 - node_modules
 - public
 - src
 - assets
 - hero.png
 - react.svg
 - vite.svg
 - components
 - Dashboard.jsx (selected)
 - Features.jsx
 - Footer.jsx
 - Hero.jsx
 - Navbar.jsx
 - Sidebar.jsx
 - SummaryCards.jsx
 - context
 - ThemeContext.jsx
- OUTLINE
- TIMELINE

The code editor shows the content of Dashboard.jsx:

```
25
26 const budgets = [
27   {
28     name: "Food",
29     spent: 72,
30   },
31   {
32     name: "Transport",
33     spent: 45,
34   },
35   {
36     name: "Entertainment",
37     spent: 58,
38   },
39 ];
40
41 const categories = [
42   {
43     name: "Housing",
44     value: "$1,200",
45   },
46   {
47     name: "Food",
48     value: "$540",
49   },
50   {
51     name: "Bills",
52     value: "$290",
53   },
54 ]
```

Screenshots:

